

Kauno technologijos universitetas
Informatikos fakultetas

Objektinis programavimas 2 (P175B123)

Laboratorinių darbų ataskaita

Karolis Žilinskas IFF 3-10

Studentas

Prof. Alfonsas Misevičius

Dėstytojas

TURINYS

1. Rekursija (L1).....	4
1.1. Darbo užduotis	4
1.2. Grafinės naudotojo sąsajos schema	4
1.3. Sąsajoje panaudotų komponentų keičiamos savybės	4
1.4. Klasių diagrama.....	6
1.5. Programos naudotojo vadovas.....	6
1.6. Programos tekstas.....	6
1.7. Pradiniai duomenys ir rezultatai	24
1.8. Dėstytojo pastabos.....	25
2. Dinaminis atminties valdymas (L2).....	26
2.1. Darbo užduotis	26
2.2. Grafinės naudotojo sąsajos schema	26
2.3. Sąsajoje panaudotų komponentų keičiamos savybės	26
2.4. Klasių diagrama.....	26
2.5. Programos naudotojo vadovas.....	26
2.6. Programos tekstas.....	26
2.7. Pradiniai duomenys ir rezultatai	26
2.8. Dėstytojo pastabos.....	27
3. Bendrinės klasės ir testavimas (L3).....	28
3.1. Darbo užduotis	28
3.2. Grafinės naudotojo sąsajos schema	28
3.3. Sąsajoje panaudotų komponentų keičiamos savybės	28
3.4. Klasių diagrama.....	28
3.5. Programos naudotojo vadovas.....	28
3.6. Programos tekstas.....	28
3.7. Pradiniai duomenys ir rezultatai	28

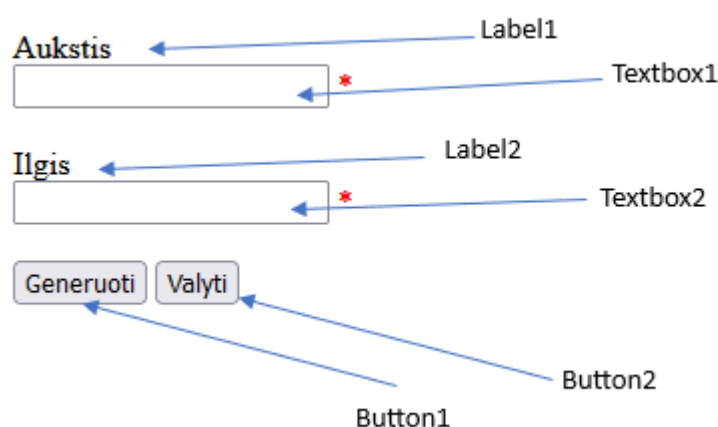
3.8.	Dėstytojo pastabos.....	29
4.	Polimorfizmas ir išimčių valdymas (L4).....	30
4.1.	Darbo užduotis	30
4.2.	Grafinės naudotojo sąsajos schema	30
4.3.	Sąsajoje panaudotų komponentų keičiamos savybės	30
4.4.	Klasių diagrama.....	30
4.5.	Programos naudotojo vadovas.....	30
4.6.	Programos tekstas.....	30
4.7.	Pradiniai duomenys ir rezultatai.....	30
4.8.	Dėstytojo pastabos.....	31
5.	Deklaratyvusis programavimas (L5).....	32
5.1.	Darbo užduotis	32
5.2.	Grafinės naudotojo sąsajos schema	32
5.3.	Sąsajoje panaudotų komponentų keičiamos savybės	32
5.4.	Klasių diagrama.....	32
5.5.	Programos naudotojo vadovas.....	32
5.6.	Programos tekstas.....	32
5.7.	Pradiniai duomenys ir rezultatai.....	32
5.8.	Dėstytojo pastabos.....	33

1. Rekursija (L1)

1.1. Darbo užduotis

Turime daug vienodo dydžio kubelių, kurių kiekvienas šonas nudažytas kokia nors spalva: r (raudona), z (žalia), g (geltona). Kubeliai paberiami ant stalo ir sustumdomi taip, kad gulėtų vienu sluoksniu ir sudarytų stačiakampį $N \times M$. Padaryti kubelių generatorių, kuris nuspalvotų kubelius ir juos sudėtų ant stalo $N \times M$ stačiakampiame plote. Čia N – eilučių skaičius, o M – stulpelių skaičius. Kubelių iš viso turi būti $N \times M$. Čia $1 \leq N \leq 20$ ir $1 \leq M \leq 30$. N ir M įvedami klaviatūra. Surasti didžiausią vienos spalvos plotą, kuriam priklauso visi kaimyniniai kubeliai, susieti ta pačia spalva viršuje ir apačioje. Kubelio kaimynu laikomas tas kubelis, kuris su juo liečiasi bent vienu tašku. Viršutinis spalvotas plotas ir apatinis spalvotas plotas laikomi vienu plotu, jeigu turi bent vieną kubelį, kurio viršus ir apačia tos pačios nagrinėjamos spalvos. Rezultatuose reikia parodyti vieną šalia kitos viršutinę ir apatinę stačiakampio puses. Surasto didžiausio vienos spalvos ploto langelius pažymėti žvaigždute *. Apačioje parašyti, iš kiek langelių sudarytas tas plotas: atskirai viršuje ir apačioje. Parašyti bent vieno kubelio, jungiančio tuos plotus (ta pati spalva viršuje ir apačioje) koordinates: eilutės numerį ir stulpelio numerį.

1.2. Grafinės naudotojo sąsajos schema

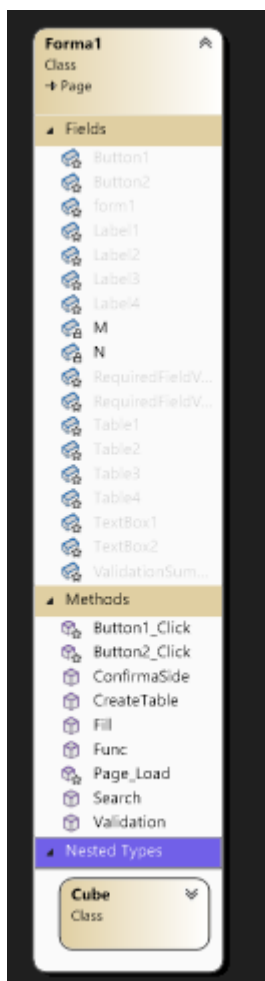


1.3. Sąsajoje panaudotų komponentų keičiamos savybės

Komponentas	Savybė	Reikšmė
ValidationSummary1	runat	server

Label3	ForeColor, Text, Visible	Red, netinkami matmenys false
Label1	Text	Aukstis
TextBox1	runat	server
RequiredFieldValidator1	ControlToValidate, ErrorMessage, ForeColor	TextBox1, iveskite auksti Red
Label2	Text	ilgis
TextBox2	runat	server
RequiredFieldValidator2	ControlToValidate, ErrorMessage, ForeColor	TextBox2, iveskite ilgi Red
Button1	OnClick, Text	Button1_Click Generuoti
Button2	OnClick, Text	Button2_Click Valyti
Table4	runat	server
Table3	runat	server
Table1	BorderColor, BorderStyle, GridLines	Black Solid Both
Table2	BorderColor, BorderStyle, GridLines	Black Solid Both
Label4	Text, Visibility	Label false

1.4. Klasių diagrama



1.5. Programos naudotojo vadovas

Į pirmą puslapio pateiktą eilutę įrašomas lentelės aukštis $1 \leq N \leq 20$, antroje – ilgis $1 \leq M \leq 30$. Paspaudus mygtuką „Generuoti“ bus sukuriamos 2 NxM dydžio lentelės, kairėje pusėje viršutinė, o apačioje apatinė, ir didžiausias vienodos spalvos kubų plotas bus pažymėtas „*“ ženklais. Paspaudus mygtuką „Valyti“ lentelės bus panaikinamos.

1.6. Programos tekstas

Forma1.aspx

```
<%@ Page Language="C#" AutoEventWireup="true" CodeBehind="Forma1.aspx.cs"
Inherits="L1_17.Forma1" %>

<!DOCTYPE html>

<html xmlns="http://www.w3.org/1999/xhtml">
<head runat="server">
```

```

        <title></title>
</head>
<body>
    <form id="form1" runat="server">
        <div style="height: 594px">

            <asp:ValidationSummary ID="ValidationSummary1" runat="server" />
            <asp:Label ID="Label3" runat="server" ForeColor="Red" Text="netinkami
matmenys" Visible="False"></asp:Label>
            <br />
            <asp:Label ID="Label1" runat="server" Text="Aukstis"></asp:Label>
            <br />
            <asp:TextBox ID="TextBox1" runat="server"></asp:TextBox>
            <asp:RequiredFieldValidator ID="RequiredFieldValidator1"
runat="server" ControlToValidate="TextBox1" ErrorMessage="iveskite auksti"
ForeColor="Red">*</asp:RequiredFieldValidator>
            <br />
            <br />
            <asp:Label ID="Label2" runat="server" Text="Ilgis"></asp:Label>
            <br />
            <asp:TextBox ID="TextBox2" runat="server"></asp:TextBox>
            <asp:RequiredFieldValidator ID="RequiredFieldValidator2"
runat="server" ControlToValidate="TextBox2" ErrorMessage="iveskite ilgi"
ForeColor="Red">*</asp:RequiredFieldValidator>
            <br />
            <br />
            <asp:Button ID="Button1" runat="server" OnClick="Button1_Click"
Text="Generuoti" />
            <asp:Button ID="Button2" runat="server" OnClick="Button2_Click"
Text="Valyti" />

            <asp:Table runat="server" ID="Table4"></asp:Table>
            <br />
            <asp:Table ID="Table3" runat="server">
                <asp:TableRow runat="server">
                    <asp:TableCell runat="server"> <asp:Table ID="Table1"
runat="server" BorderColor="Black" BorderStyle="Solid" GridLines="Both">
</asp:Table></asp:TableCell>
                    <asp:TableCell runat="server"></asp:TableCell>
                    <asp:TableCell runat="server"> <asp:Table ID="Table2"
runat="server" BorderColor="Black" BorderStyle="Solid" GridLines="Both">
</asp:Table></asp:TableCell>
                </asp:TableRow>
            </asp:Table>
            <br />
            <asp:Label ID="Label4" runat="server" Text="Label"
Visible="False"></asp:Label>

            <br />

            &nbsp; &nbsp; &nbsp; &nbsp;

        </div>
    </form>
</body>
</html>

```

Forma1.aspx.cs

```

using System;
using System.Collections.Generic;
using System.Drawing;
using System.Text.RegularExpressions;

```

```

using System.Web.UI.WebControls;
using System.Xml.Schema;

namespace L1_17
{
    public partial class Formal : System.Web.UI.Page
    {
        /// <summary>
        /// N(height) and M(length) of the tables
        /// </summary>
        private int N;
        private int M;

        /// <summary>
        /// cubes with a "checked" status
        /// </summary>
        public class Cube
        {
            public bool Checked { get; set; }
            public Cube()
            {
                Checked = false;
            }
        }

        /// <summary>
        /// page load
        /// </summary>
        /// <param name="sender"></param>
        /// <param name="e"></param>
        protected void Page_Load(object sender, EventArgs e)
        {

        }

        /// <summary>
        /// Resets the tables
        /// </summary>
        /// <param name="sender"></param>
        /// <param name="e"></param>
        protected void Button2_Click(object sender, EventArgs e)
        {
            Label4.Visible = false;
        }

        /// <summary>
        /// creates 2 tables and finds the biggest patch of same color connected
cubes
        /// </summary>
        /// <param name="sender"></param>
        /// <param name="e"></param>
        protected void Button1_Click(object sender, EventArgs e)
        {
            Label3.Visible = false;
            if (Validation())
            {
                Label3.Visible = true;
                return;
            }

            N = int.Parse(TextBox1.Text);
            M = int.Parse(TextBox2.Text);

            Random random = new Random();

```



```

List<List<Cube>> Cubes1 = new List<List<Cube>>();
CreateTable(Table1, Cubes1, random);
List<List<Cube>> Cubes2 = new List<List<Cube>>();
CreateTable(Table2, Cubes2, random);

int MaxI = -1;
int MaxJ = -1;
int MaxBlob = 0;

Cube TopCube = new Cube();

Func(true, Cubes1, Cubes2, Table1, Table2, ref MaxI, ref MaxJ, ref MaxBlob);

//-----
for (int ii = 0; ii < N; ii++)
{
    for (int jj = 0; jj < M; jj++)
    {
        Cubes1[ii][jj].Checked = false;
        Cubes2[ii][jj].Checked = false;
    }
}
//-----

bool StartSide = ConfirmaSide(MaxI, MaxJ, Cubes1, Cubes2, Table1,
Table2);

//-----
for (int ii = 0; ii < N; ii++)
{
    for (int jj = 0; jj < M; jj++)
    {
        Cubes1[ii][jj].Checked = false;
        Cubes2[ii][jj].Checked = false;
    }
}
//-----

int ConnectorI = -1; int ConnectorJ = -1;

Fill(MaxI, MaxJ, true, Cubes1, Cubes2, "", ref ConnectorI, ref ConnectorJ, ref
StartSide);

//-----
int MaxTop = 0;
int MaxBottom = 0;
MaxBlob = 0;
for (int ii = 0; ii < N; ii++)
{
    for (int jj = 0; jj < M; jj++)
    {
        if (Table1.Rows[ii].Cells[jj].Text != "aaa") MaxTop++;
        if (Table2.Rows[ii].Cells[jj].Text != "aaa") MaxBottom++;
    }
}
MaxBlob = MaxTop + MaxBottom;
//-----

Label4.Visible = true;
Label4.Text = "Didžiausią plotą sudaro viršuje " + MaxTop + " ir
apačioje " + MaxBottom + " langelių. Langelis: " + (ConnectorI+1) + " eil.,
" + (ConnectorJ+1) + " st";

```

```

    }
    /// <summary>
    /// validates if the submitted data is usable
    /// </summary>
    /// <returns></returns>
    public bool Validation()
    {
        if (Regex.IsMatch(TextBox1.Text.ToLower(), "[a-začĕėišųūž]") ||
Regex.IsMatch(TextBox2.Text.ToLower(), "[a-začĕėišųūž]"))
        {
            return true;
        }
        else if (int.Parse(TextBox1.Text) <= 0 || int.Parse(TextBox1.Text) >
20 || int.Parse(TextBox2.Text) <= 0 || int.Parse(TextBox2.Text) > 30) return true;

        return false;
    }
    /// <summary>
    /// Creates a table full of random color cubes
    /// </summary>
    /// <param name="table"></param>
    /// <param name="cubes"></param>
    /// <param name="random"></param>
    /// <returns></returns>
    public Table CreateTable(Table table, List<List<Cube>> cubes, Random
random)
    {

        for (int i = 0; i < N; i++)
        {
            TableRow row = new TableRow();
            List<Cube> list = new List<Cube>();
            for (int j = 0; j < M; j++)
            {
                TableCell cell = new TableCell();
                cell.Text = "aaa";

                Cube cube = new Cube();

                int num = random.Next(1, 4);
                switch (num)
                {
                    case 1:

                        cell.BackColor = Color.Red;
                        cell.ForeColor = Color.Red;

                        cube = new Cube();

                        break;
                    case 2:
                        cell.BackColor = Color.Yellow;
                        cell.ForeColor = Color.Yellow;

                        cube = new Cube();

                        break;
                    case 3:
                        cell.BackColor = Color.DarkOliveGreen;
                        cell.ForeColor = Color.DarkOliveGreen;

```

```

        cube = new Cube();

        break;
    }
    cell.BorderColor = Color.Black;
    row.Cells.Add(cell);
    list.Add(cube);
}
table.Rows.Add(row);
cubes.Add(list);
}
return table;
}
/// <summary>
/// The cycle in which the recursive function will run. Finds the biggest
patch of cubes
/// </summary>
/// <param name="topside"></param>
/// <param name="Cubes1"></param>
/// <param name="Cubes2"></param>
/// <param name="Table1"></param>
/// <param name="Table2"></param>
/// <param name="maxI"></param>
/// <param name="maxJ"></param>
/// <param name="MaxBlob"></param>
public void Func(bool topside, List<List<Cube>> Cubes1, List<List<Cube>>
Cubes2, Table Table1, Table Table2, ref int maxI, ref int maxJ, ref int MaxBlob)
{
    maxI = -1;
    maxJ = -1;
    MaxBlob = 0;
    int CurrentBlob;

    string CurrentColor = "";
    for (int i = 0; i < N; i++ )
    {
        for(int j = 0; j < M; j++ )
        {
            Cubes1[i][j].Checked = true;
            CurrentColor = Table1.Rows[i].Cells[j].BackColor.ToString();
            CurrentBlob = 1;
            if (topside)
            {
                if (!Cubes1[i][j].Checked && CurrentColor ==
Table1.Rows[i].Cells[j].BackColor.ToString())
                {
                    topside = true;
                    Cubes2[i][j].Checked = true;
                    CurrentBlob++;
                    if (CurrentBlob > MaxBlob)
                    {
                        MaxBlob = CurrentBlob;
                        maxI = i;
                        maxJ = j;
                    }
                    Search(i, j, topside, Cubes1, Cubes2, CurrentColor,
ref MaxBlob, ref CurrentBlob, ref maxI, ref maxJ);
                }
            }
        }
    }
}

```

```

        if (!Cubes2[i][j].Checked && CurrentColor ==
Table2.Rows[i].Cells[j].BackColor.ToString())
        {

            topside = false;
            Cubes1[i][j].Checked = true;
            CurrentBlob++;
            if (CurrentBlob > MaxBlob)
            {
                MaxBlob = CurrentBlob;
                maxI = i;
                maxJ = j;
            }
            Search(i, j, topside, Cubes1, Cubes2, CurrentColor,
ref MaxBlob, ref CurrentBlob, ref maxI, ref maxJ);

        }

        if (j < M-1)
        {
            if (!Cubes1[i][j + 1].Checked && CurrentColor ==
Table1.Rows[i].Cells[j + 1].BackColor.ToString())
            {
                Cubes1[i][j].Checked = true;
                CurrentBlob++;
                if (CurrentBlob > MaxBlob)
                {
                    MaxBlob = CurrentBlob;
                    maxI = i;
                    maxJ = j;
                }
                Search(i, j + 1, topside, Cubes1, Cubes2,
CurrentColor, ref MaxBlob, ref CurrentBlob, ref maxI, ref maxJ);

            }

        }

        if (i < N-1)
        {
            if (!Cubes1[i + 1][j].Checked && CurrentColor ==
Table1.Rows[i + 1].Cells[j].BackColor.ToString())
            {

                Cubes1[i][j].Checked = true;
                CurrentBlob++;
                if (CurrentBlob > MaxBlob)
                {
                    MaxBlob = CurrentBlob;
                    maxI = i;
                    maxJ = j;
                }

                Search(i + 1, j, topside, Cubes1, Cubes2,
CurrentColor, ref MaxBlob, ref CurrentBlob, ref maxI, ref maxJ);

            }

        }

        if (j > 0)
        {

```

```

        if (!Cubes1[i][j - 1].Checked && CurrentColor ==
Table1.Rows[i].Cells[j - 1].BackColor.ToString())
        {
            Cubes1[i][j].Checked = true;
            CurrentBlob++;
            if (CurrentBlob > MaxBlob)
            {
                MaxBlob = CurrentBlob;
                maxI = i;
                maxJ = j;
            }
            Search(i, j - 1, topside, Cubes1, Cubes2,
CurrentColor, ref MaxBlob, ref CurrentBlob, ref maxI, ref maxJ);
        }
    }

    if (i > 0)
    {
        if (!Cubes1[i - 1][j].Checked && CurrentColor ==
Table1.Rows[i - 1].Cells[j].BackColor.ToString())
        {
            Cubes1[i][j].Checked = true;
            CurrentBlob++;
            if (CurrentBlob > MaxBlob)
            {
                MaxBlob = CurrentBlob;
                maxI = i;
                maxJ = j;
            }
            Search(i - 1, j, topside, Cubes1, Cubes2,
CurrentColor, ref MaxBlob, ref CurrentBlob, ref maxI, ref maxJ);
        }
    }

}

if (!topside)
{
    if (j < M - 1)
    {
        if (!Cubes2[i][j + 1].Checked && CurrentColor ==
Table2.Rows[i].Cells[j + 1].BackColor.ToString())
        {
            Cubes2[i][j].Checked = true;
            CurrentBlob++;
            if (CurrentBlob > MaxBlob)
            {
                MaxBlob = CurrentBlob;
                maxI = i;
                maxJ = j;
            }
            Search(i, j + 1, topside, Cubes1, Cubes2,
CurrentColor, ref MaxBlob, ref CurrentBlob, ref maxI, ref maxJ);
        }
    }
}

```

```

    }

    if (i < N - 1)
    {
        if (!Cubes2[i + 1][j].Checked && CurrentColor ==
Table2.Rows[i + 1].Cells[j].BackColor.ToString())
        {
            Cubes2[i][j].Checked = true;
            CurrentBlob++;
            if (CurrentBlob > MaxBlob)
            {
                MaxBlob = CurrentBlob;
                maxI = i;
                maxJ = j;
            }
            Search(i + 1, j, topside, Cubes1, Cubes2,
CurrentColor, ref MaxBlob, ref CurrentBlob, ref maxI, ref maxJ);
        }
    }

    if (j > 0)
    {
        if (!Cubes2[i][j - 1].Checked && CurrentColor ==
Table2.Rows[i].Cells[j - 1].BackColor.ToString())
        {
            Cubes2[i][j].Checked = true;
            CurrentBlob++;
            if (CurrentBlob > MaxBlob)
            {
                MaxBlob = CurrentBlob;
                maxI = i;
                maxJ = j;
            }
            Search(i, j - 1, topside, Cubes1, Cubes2,
CurrentColor, ref MaxBlob, ref CurrentBlob, ref maxI, ref maxJ);
        }
    }

    if (i > 0)
    {
        if (!Cubes2[i - 1][j].Checked && CurrentColor ==
Table2.Rows[i - 1].Cells[j].BackColor.ToString())
        {
            Cubes2[i][j].Checked = true;
            CurrentBlob++;
            if (CurrentBlob > MaxBlob)
            {
                MaxBlob = CurrentBlob;
                maxI = i;
                maxJ = j;
            }
            Search(i - 1, j, topside, Cubes1, Cubes2,
CurrentColor, ref MaxBlob, ref CurrentBlob, ref maxI, ref maxJ);
        }
    }
}

```

```

    }

    }

    }
    //starting from the other table incase some cubes were missed

    topside = false;
    for (int i = 0; i < N; i++)
    {

        for (int j = 0; j < M; j++)
        {

            Cubes2[i][j].Checked = true;
            CurrentColor = Table2.Rows[i].Cells[j].BackColor.ToString();
            CurrentBlob = 1;
            if (topside)
            {

                if (!Cubes2[i][j].Checked && CurrentColor ==
Table2.Rows[i].Cells[j].BackColor.ToString())
                {

                    topside = false;
                    Cubes1[i][j].Checked = true;
                    CurrentBlob++;
                    if (CurrentBlob > MaxBlob)
                    {
                        MaxBlob = CurrentBlob;
                        maxI = i;
                        maxJ = j;
                    }
                    Search(i, j, topside, Cubes1, Cubes2, CurrentColor,
ref MaxBlob, ref CurrentBlob, ref maxI, ref maxJ);
                }

                if (j < M - 1)
                {
                    if (!Cubes1[i][j + 1].Checked && CurrentColor ==
Table1.Rows[i].Cells[j + 1].BackColor.ToString())
                    {
                        Cubes1[i][j].Checked = true;
                        CurrentBlob++;
                        if (CurrentBlob > MaxBlob)
                        {
                            MaxBlob = CurrentBlob;
                            maxI = i;
                            maxJ = j;
                        }
                    }
                    Search(i, j + 1, topside, Cubes1, Cubes2,
CurrentColor, ref MaxBlob, ref CurrentBlob, ref maxI, ref maxJ);
                }

            }

            if (i < N - 1)
            {
                if (!Cubes1[i + 1][j].Checked && CurrentColor ==
Table1.Rows[i + 1].Cells[j].BackColor.ToString())

```

```

        {
            Cubes1[i][j].Checked = true;
            CurrentBlob++;
            if (CurrentBlob > MaxBlob)
            {
                MaxBlob = CurrentBlob;
                maxI = i;
                maxJ = j;
            }
            Search(i + 1, j, topside, Cubes1, Cubes2,
CurrentColor, ref MaxBlob, ref CurrentBlob, ref maxI, ref maxJ);
        }
    }

    if (j > 0)
    {
        if (!Cubes1[i][j - 1].Checked && CurrentColor ==
Table1.Rows[i].Cells[j - 1].BackColor.ToString())
        {
            Cubes1[i][j].Checked = true;
            CurrentBlob++;
            if (CurrentBlob > MaxBlob)
            {
                MaxBlob = CurrentBlob;
                maxI = i;
                maxJ = j;
            }
            Search(i, j - 1, topside, Cubes1, Cubes2,
CurrentColor, ref MaxBlob, ref CurrentBlob, ref maxI, ref maxJ);
        }
    }

    if (i > 0)
    {
        if (!Cubes1[i - 1][j].Checked && CurrentColor ==
Table1.Rows[i - 1].Cells[j].BackColor.ToString())
        {
            Cubes1[i][j].Checked = true;
            CurrentBlob++;
            if (CurrentBlob > MaxBlob)
            {
                MaxBlob = CurrentBlob;
                maxI = i;
                maxJ = j;
            }
            Search(i - 1, j, topside, Cubes1, Cubes2,
CurrentColor, ref MaxBlob, ref CurrentBlob, ref maxI, ref maxJ);
        }
    }
}

if (!topside)
{

```



```

        if (!Cubes1[i][j].Checked && CurrentColor ==
Table1.Rows[i].Cells[j].BackColor.ToString())
        {

            topside = true;
            Cubes2[i][j].Checked = true;
            CurrentBlob++;
            if (CurrentBlob > MaxBlob)
            {
                MaxBlob = CurrentBlob;
                maxI = i;
                maxJ = j;
            }
            Search(i, j, topside, Cubes1, Cubes2, CurrentColor,
ref MaxBlob, ref CurrentBlob, ref maxI, ref maxJ);

        }
        if (j < M - 1)
        {
            if (!Cubes2[i][j + 1].Checked && CurrentColor ==
Table2.Rows[i].Cells[j + 1].BackColor.ToString())
            {

                Cubes2[i][j].Checked = true;
                CurrentBlob++;
                if (CurrentBlob > MaxBlob)
                {
                    MaxBlob = CurrentBlob;
                    maxI = i;
                    maxJ = j;
                }
                Search(i, j + 1, topside, Cubes1, Cubes2,
CurrentColor, ref MaxBlob, ref CurrentBlob, ref maxI, ref maxJ);

            }
        }

        if (i < N - 1)
        {
            if (!Cubes2[i + 1][j].Checked && CurrentColor ==
Table2.Rows[i + 1].Cells[j].BackColor.ToString())
            {

                Cubes2[i][j].Checked = true;
                CurrentBlob++;
                if (CurrentBlob > MaxBlob)
                {
                    MaxBlob = CurrentBlob;
                    maxI = i;
                    maxJ = j;
                }
                Search(i + 1, j, topside, Cubes1, Cubes2,
CurrentColor, ref MaxBlob, ref CurrentBlob, ref maxI, ref maxJ);

            }
        }

        if (j > 0)
        {
            if (!Cubes2[i][j - 1].Checked && CurrentColor ==
Table2.Rows[i].Cells[j - 1].BackColor.ToString())
            {

```



```

        {
            MaxBlob = CurrentBlob;
            maxI = i;
            maxJ = j;
        }
        if (!Cubes2[i][j].Checked && CurrentColor ==
Table2.Rows[i].Cells[j].BackColor.ToString() &&
Table2.Rows[i].Cells[j].BackColor.ToString() ==
Table1.Rows[i].Cells[j].BackColor.ToString())
        {
            topside = false;

            Search(i, j, topside, Cubes1, Cubes2, CurrentColor, ref
MaxBlob, ref CurrentBlob, ref maxI, ref maxJ);

        }

        if (j < M - 1)
        {
            if (!Cubes1[i][j + 1].Checked && CurrentColor ==
Table1.Rows[i].Cells[j + 1].BackColor.ToString())
            {
                topside = true;
                Search(i, j + 1, topside, Cubes1, Cubes2, CurrentColor,
ref MaxBlob, ref CurrentBlob, ref maxI, ref maxJ);

            }
        }

        if (i < N - 1)
        {
            if (!Cubes1[i + 1][j].Checked && CurrentColor == Table1.Rows[i
+ 1].Cells[j].BackColor.ToString())
            {
                topside = true;
                Search(i + 1, j, topside, Cubes1, Cubes2, CurrentColor,
ref MaxBlob, ref CurrentBlob, ref maxI, ref maxJ);

            }
        }

        if (j > 0)
        {
            if (!Cubes1[i][j - 1].Checked && CurrentColor ==
Table1.Rows[i].Cells[j - 1].BackColor.ToString())
            {
                topside = true;
                Search(i, j - 1, topside, Cubes1, Cubes2, CurrentColor,
ref MaxBlob, ref CurrentBlob, ref maxI, ref maxJ);

            }
        }

        if (i > 0)
        {
            if (!Cubes1[i - 1][j].Checked && CurrentColor == Table1.Rows[i
- 1].Cells[j].BackColor.ToString())
            {
                topside = true;

```

```

        Search(i - 1, j, topside, Cubes1, Cubes2, CurrentColor,
ref MaxBlob, ref CurrentBlob, ref maxI, ref maxJ);

    }

}

}
if (!topside)
{
    Cubes2[i][j].Checked = true;
    CurrentBlob++;
    if (CurrentBlob > MaxBlob)
    {
        MaxBlob = CurrentBlob;
        maxI = i;
        maxJ = j;
    }
    if (!Cubes1[i][j].Checked && CurrentColor ==
Table1.Rows[i].Cells[j].BackColor.ToString() &&
Table2.Rows[i].Cells[j].BackColor.ToString() ==
Table1.Rows[i].Cells[j].BackColor.ToString())
    {
        topside = true;

        Search(i, j, topside, Cubes1, Cubes2, CurrentColor, ref
MaxBlob, ref CurrentBlob, ref maxI, ref maxJ);

    }

    if (j < M - 1)
    {
        if (!Cubes2[i][j + 1].Checked && CurrentColor ==
Table2.Rows[i].Cells[j + 1].BackColor.ToString())
        {

            topside = false;
            Search(i, j + 1, topside, Cubes1, Cubes2, CurrentColor,
ref MaxBlob, ref CurrentBlob, ref maxI, ref maxJ);

        }
    }

    if (i < N - 1)
    {
        if (!Cubes2[i + 1][j].Checked && CurrentColor == Table2.Rows[i
+ 1].Cells[j].BackColor.ToString())
        {

            topside = false;
            Search(i + 1, j, topside, Cubes1, Cubes2, CurrentColor,
ref MaxBlob, ref CurrentBlob, ref maxI, ref maxJ);

        }
    }

    if (j > 0)
    {
        if (!Cubes2[i][j - 1].Checked && CurrentColor ==
Table2.Rows[i].Cells[j - 1].BackColor.ToString())
        {

            topside = false;

```

```

        Search(i, j - 1, topside, Cubes1, Cubes2, CurrentColor,
ref MaxBlob, ref CurrentBlob, ref maxI, ref maxJ);

    }
}

    if (i > 0)
    {
        if (!Cubes2[i - 1][j].Checked && CurrentColor == Table2.Rows[i
- 1].Cells[j].BackColor.ToString())
        {

            topside = false;
            Search(i - 1, j, topside, Cubes1, Cubes2, CurrentColor,
ref MaxBlob, ref CurrentBlob, ref maxI, ref maxJ);

        }
    }

}

/// <summary>
/// Recursive function. Fills in the the cubes of the biggest patch with
"""
/// </summary>
/// <param name="i"></param>
/// <param name="j"></param>
/// <param name="topside"></param>
/// <param name="Cubes1"></param>
/// <param name="Cubes2"></param>
/// <param name="CurrentColor"></param>
/// <param name="connectorI"></param>
/// <param name="connectorJ"></param>
/// <param name="StartSide"></param>
public void Fill(int i, int j, bool topside, List<List<Cube>> Cubes1,
List<List<Cube>> Cubes2, string CurrentColor, ref int connectorI, ref int
connectorJ, ref bool StartSide)
{
    if (StartSide && CurrentColor == "")
    {
        topside = true;
        Table1.Rows[i].Cells[j].Text = " * ";
        Table1.Rows[i].Cells[j].ForeColor = Color.Black;
        CurrentColor = Table1.Rows[i].Cells[j].BackColor.ToString();
    }
    else if (!StartSide && CurrentColor == "")
    {
        topside = false;
        Table2.Rows[i].Cells[j].Text = " * ";
        Table2.Rows[i].Cells[j].ForeColor = Color.Black;
        CurrentColor = Table2.Rows[i].Cells[j].BackColor.ToString();
    }

    if (topside)
    {
        Table1.Rows[i].Cells[j].Text = " * ";
        Table1.Rows[i].Cells[j].ForeColor = Color.Black;
        Cubes1[i][j].Checked = true;

        if (!Cubes2[i][j].Checked && CurrentColor ==
Table2.Rows[i].Cells[j].BackColor.ToString() &&

```

```

Table2.Rows[i].Cells[j].BackColor.ToString() ==
Table1.Rows[i].Cells[j].BackColor.ToString())
    {
        topside = false;

        connectorI = i;
        connectorJ = j;
        Fill(i, j, topside, Cubes1, Cubes2, CurrentColor, ref
connectorI, ref connectorJ, ref StartSide);

    }

    if (j < M - 1)
    {
        if (!Cubes1[i][j + 1].Checked && CurrentColor ==
Table1.Rows[i].Cells[j + 1].BackColor.ToString())
        {
            topside = true;
            Fill(i, j + 1, topside, Cubes1, Cubes2, CurrentColor, ref
connectorI, ref connectorJ, ref StartSide);

        }
    }

    if (i < N - 1)
    {
        if (!Cubes1[i + 1][j].Checked && CurrentColor == Table1.Rows[i
+ 1].Cells[j].BackColor.ToString())
        {
            topside = true;
            Fill(i + 1, j, topside, Cubes1, Cubes2, CurrentColor, ref
connectorI, ref connectorJ, ref StartSide);

        }
    }

    if (j > 0)
    {
        if (!Cubes1[i][j - 1].Checked && CurrentColor ==
Table1.Rows[i].Cells[j - 1].BackColor.ToString())
        {
            topside = true;
            Fill(i, j - 1, topside, Cubes1, Cubes2, CurrentColor, ref
connectorI, ref connectorJ, ref StartSide);

        }
    }

    if (i > 0)
    {
        if (!Cubes1[i - 1][j].Checked && CurrentColor == Table1.Rows[i
- 1].Cells[j].BackColor.ToString())
        {
            topside = true;
            Fill(i - 1, j, topside, Cubes1, Cubes2, CurrentColor, ref
connectorI, ref connectorJ, ref StartSide);

        }
    }
}

```

```

    }
    if (!topside)
    {
        Table2.Rows[i].Cells[j].Text = " * ";
        Table2.Rows[i].Cells[j].ForeColor = Color.Black;
        Cubes2[i][j].Checked = true;
        if (!Cubes1[i][j].Checked && CurrentColor ==
Table1.Rows[i].Cells[j].BackColor.ToString() &&
Table2.Rows[i].Cells[j].BackColor.ToString() ==
Table1.Rows[i].Cells[j].BackColor.ToString())
        {
            topside = true;

            connectorI = i;
            connectorJ = j;
            Fill(i, j, topside, Cubes1, Cubes2, CurrentColor, ref
connectorI, ref connectorJ, ref StartSide);
        }

        if (j < M - 1)
        {
            if (!Cubes2[i][j + 1].Checked && CurrentColor ==
Table2.Rows[i].Cells[j + 1].BackColor.ToString())
            {
                topside = false;
                Fill(i, j + 1, topside, Cubes1, Cubes2, CurrentColor, ref
connectorI, ref connectorJ, ref StartSide);
            }
        }

        if (i < N - 1)
        {
            if (!Cubes2[i + 1][j].Checked && CurrentColor == Table2.Rows[i
+ 1].Cells[j].BackColor.ToString())
            {
                topside = false;
                Fill(i + 1, j, topside, Cubes1, Cubes2, CurrentColor, ref
connectorI, ref connectorJ, ref StartSide);
            }
        }

        if (j > 0)
        {
            if (!Cubes2[i][j - 1].Checked && CurrentColor ==
Table2.Rows[i].Cells[j - 1].BackColor.ToString())
            {
                topside = false;
                Fill(i, j - 1, topside, Cubes1, Cubes2, CurrentColor, ref
connectorI, ref connectorJ, ref StartSide);
            }
        }

        if (i > 0)

```

```

        {
            if (!Cubes2[i - 1][j].Checked && CurrentColor == Table2.Rows[i
- 1].Cells[j].BackColor.ToString())
            {
                topside = false;
                Fill(i - 1, j, topside, Cubes1, Cubes2, CurrentColor, ref
connectorI, ref connectorJ, ref StartSide);
            }
        }

    }

}

/// <summary>
/// Finds if the start of the search for a cube patch begins from the top
or bottom table
/// </summary>
/// <param name="i"></param>
/// <param name="j"></param>
/// <param name="Cubes1"></param>
/// <param name="Cubes2"></param>
/// <param name="Table1"></param>
/// <param name="Table2"></param>
/// <returns></returns>
public bool ConfirmaSide(int i, int j, List<List<Cube>> Cubes1,
List<List<Cube>> Cubes2, Table Table1, Table Table2)
{
    if (Table1.Rows[i].Cells[j].BackColor ==
Table2.Rows[i].Cells[j].BackColor)
        return true;

    int CurrentBlobTop = 1;
    int CurrentBlobBottom=1;
    int MaxBlobTop = 1;
    int MaxBlobBottom = 1;
    int MaxITop = 0;
    int MaxIBottom = 0;
    int MaxJTop = 0;
    int MaxJBottom = 0;

    Search(i, j, true, Cubes1, Cubes2,
Table1.Rows[i].Cells[j].BackColor.ToString(), ref MaxBlobTop, ref CurrentBlobTop, ref
MaxITop, ref MaxJTop);
    Search(i, j, false, Cubes1, Cubes2,
Table2.Rows[i].Cells[j].BackColor.ToString(), ref MaxBlobBottom, ref
CurrentBlobBottom, ref MaxIBottom, ref MaxJBottom);

    return MaxBlobTop > MaxBlobBottom;
}
}
}

```

1.7. Pradiniai duomenys ir rezultatai

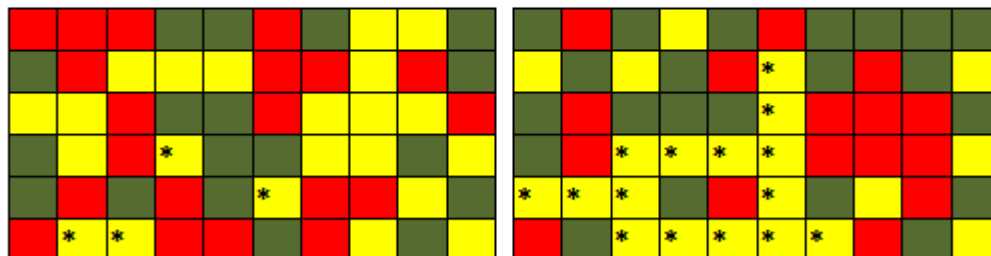
Įprastas programos veikimas:

Aukstis

Ilgis

Generuoti

Valyti



Didžiausią plotą sudaro viršuje 4 ir apačioje 15 langelių. Langelis: 4 eil., 4 st

Programos veikimas, kai vedami netinkami duomenys:

- iveskite ilgi

netinkami matmenys

Aukstis

Ilgis

Generuoti

Valyti

1.8. Dėstytojo pastabos

2. Dinaminis atminties valdymas (L2)

2.1. Darbo užduotis

2.2. Grafinės naudotojo sąsajos schema

2.3. Sąsajoje panaudotų komponentų keičiamos savybės

Komponentas	Savybė	Reikšmė

2.4. Klasių diagrama

2.5. Programos naudotojo vadovas

2.6. Programos tekstas

2.7. Pradiniai duomenys ir rezultatai

2.8. Dėstytojo pastabos

3. Bendrinės klasės ir testavimas (L3)

3.1. Darbo užduotis

3.2. Grafinės naudotojo sąsajos schema

3.3. Sąsajoje panaudotų komponentų keičiamos savybės

Komponentas	Savybė	Reikšmė

3.4. Klasių diagrama

3.5. Programos naudotojo vadovas

3.6. Programos tekstas

3.7. Pradiniai duomenys ir rezultatai

3.8. Dėstytojo pastabos

4. Polimorfizmas ir išimčių valdymas (L4)

4.1. Darbo užduotis

4.2. Grafinės naudotojo sąsajos schema

4.3. Sąsajoje panaudotų komponentų keičiamos savybės

Komponentas	Savybė	Reikšmė

4.4. Klasių diagrama

4.5. Programos naudotojo vadovas

4.6. Programos tekstas

4.7. Pradiniai duomenys ir rezultatai

4.8. Dėstytojo pastabos

5. Deklaratyvusis programavimas (L5)

5.1. Darbo užduotis

5.2. Grafinės naudotojo sąsajos schema

5.3. Sąsajoje panaudotų komponentų keičiamos savybės

Komponentas	Savybė	Reikšmė

5.4. Klasių diagrama

5.5. Programos naudotojo vadovas

5.6. Programos tekstas

5.7. Pradiniai duomenys ir rezultatai

5.8. Dėstytojo pastabos